

Universidad ORT
Facultad de Ingeniería

Obligatorio 1 - Diseño de Aplicaciones 1
Ingeniería en Sistemas

Martina Barone - 302285

Inés Gayol - 281635

Ana Clara Vázquez - 282326

Profesores: Juan Ignacio Díaz, Santiago Pérez y Luis Eduardo Simón

2025

Link al repositorio de la entrega:

https://github.com/IngSoft-DA1/302285_281635_282326

Índice

1. Descripción general.....	4
2. Diseño.....	4
3. Diagrama de paquetes.....	7
4. Diagramas de clases.....	8
4.1 Dominio.....	8
4.2 Servicios.....	10
4.3 Repositorios.....	11
4. Dependencias generadas entre Interfaz Gráfica y Lógica de Negocio.....	12
5. Explicación de validaciones de negocio.....	13
6. Mecanismos generales y descripción de las principales decisiones de diseño tomadas....	16
7. Mantenibilidad y extensibilidad.....	17
8. Algoritmo de Camino Crítico y evidencia de TDD.....	18
9. Cobertura de pruebas unitarias.....	18
10. Evidencia de TDD en la clase Tarea.....	18
11. Casos de prueba elaborados.....	18
12. Referencias bibliográficas.....	19
 Anexo 1. Diagramas UML.....	 20
1.1 Diagrama de Paquetes	20
1.2 Dominio.....	21
1.3 Servicios.....	21
1.4 Repositorios.....	22
Anexo 2. Algoritmo de Camino Crítico y evidencia de TDD.....	23
Anexo 3. Cobertura de pruebas unitarias.....	28
Anexo 4. Evidencia de TDD en la clase Tarea.....	29
Anexo 5. Librerías externas.....	30
Anexo 6. Bosquejo de UI.....	31
Anexo 7. Datos precargados.....	32

Descripción general

comportamientos variables dentro del dominio. El control de versiones y el trabajo Este trabajo consiste en el desarrollo de un sistema de gestión de proyectos llamado **TaskTrackPro**, el cual permite la administración colaborativa de tareas, recursos y dependencias entre actividades dentro de un proyecto.

La aplicación fue construida utilizando el lenguaje C# bajo el framework .NET 8, implementada como una Blazor Server App, asegurando una experiencia web interactiva con renderizado en servidor. Se aplicó la metodología *Test Driven Development (TDD)* durante el diseño e implementación de la lógica del sistema, utilizando el framework de pruebas unitarias MSTest. Además, se siguieron los principios de *Clean Code*, con especial atención a la claridad, simplicidad y responsabilidad única en cada clase. Se adoptaron prácticas de *GRASP* para asignar responsabilidades, y se utilizó *polimorfismo* para encapsular colaborativo se gestionaron mediante GitHub, siguiendo estrictamente la metodología *Gitflow*.

Todas las funcionalidades requeridas en la consigna han sido implementadas y validadas mediante pruebas unitarias automatizadas. La cobertura de los tests es de 100% y se puede evidenciar el uso de TDD en los commits de cada una de las clases.

Diseño

Namespace	Clase	Responsabilidad
Dominio	Usuario	Representa a un usuario del sistema, es responsable de integridad de sus propios datos y comportamientos asociados a su rol en la aplicación.
Dominio	Proyecto	Representa un proyecto. Administra sus tareas, miembros, fechas clave, notificaciones, validaciones relacionadas y modifica sus datos. Además, gestiona el vínculo entre los usuarios y sus roles dentro del proyecto.
Dominio	Tarea	Representa una tarea, validando sus datos. Gestiona dependencias, miembros y recursos asignados. Controla los cambios de estado, calcula la fecha de finalización más temprana, determina si la tarea es crítica y permite modificar sus atributos.

Dominio	Recurso	Representa un recurso, validando sus datos. Gestiona su uso dentro de tareas, su posible asociación exclusiva a un proyecto y permite modificar sus atributos.
Dominio	Notificación	Representa una notificación del sistema. Asigna automáticamente un identificador único, valida que el mensaje no sea vacío y registra la fecha de creación.
Dominio	Dependencia	Representa una dependencia entre tareas. Valida que el tipo sea correcto y que la tarea asociada no sea nula. Permite comparar dependencias por tipo y tarea para evitar duplicaciones.
Dominio	EstadoTarea	Define los posibles estados que puede tener una tarea.
Servicios.Excepciones	ExcepcionesServicios	Representar errores específicos de la capa de servicios.
Servicios.Gestores	GestorUsuarios	Gestiona el ciclo de vida de los usuarios, incluyendo su creación, eliminación, búsqueda y modificación de roles. Valida reglas de negocio relacionadas con permisos, contraseñas y asignaciones a proyectos. Además, administra notificaciones ante cambios relevantes en la cuenta de un usuario.
Servicios.Gestores	GestorProyectos	Administra la creación, eliminación y modificación de proyectos, incluyendo sus atributos principales como nombre, descripción y fecha de inicio. Gestiona los miembros del proyecto, permitiendo agregarlos, eliminarlos o cambiar al administrador. Asegura que se respeten las reglas del negocio mediante validaciones de permisos y estados, y permite consultar los proyectos relacionados a un usuario.
Servicios.Gestores	GestorTareas	Administra la creación, eliminación y actualización de tareas dentro de un proyecto. Permite modificar atributos como título, descripción, duración, fecha de inicio y estado. Gestiona

		dependencias, recursos y usuarios asignados a cada tarea. Valida los permisos y reglas de negocio antes de ejecutar cambios, recalcula el camino crítico ante modificaciones relevantes y notifica a los miembros del proyecto.
Servicios.Gestores	GestorRecursos	Administra la creación, eliminación y edición de recursos dentro del sistema. Permite modificar atributos como nombre, tipo y descripción, y asocia recursos exclusivos al proyecto correspondiente. Verifica que solo los usuarios con permisos adecuados puedan operar sobre cada recurso, diferenciando entre recursos generales y exclusivos. Además, controla si un recurso está en uso antes de permitir su eliminación o modificación, y notifica a los administradores involucrados ante cualquier cambio relevante.
Servicios.Utilidades	CaminoCritico	Calcula el camino crítico de un proyecto a partir de sus tareas, considerando sus dependencias del tipo Fin-Inicio (FS) e Inicio-Inicio (SS). Ordena las tareas topológicamente para validar la ausencia de ciclos, determina las fechas más tempranas de inicio y fin, y calcula la holgura de cada tarea. Notifica a los miembros del proyecto si se modifica la fecha de finalización más temprana.
Servicios.Utilidades	UtilidadesContrasena	Valida que una contraseña cumpla con los requisitos mínimos de seguridad definidos: longitud, inclusión de letras mayúsculas, minúsculas, números y caracteres especiales. Además, permite encriptar contraseñas utilizando el algoritmo BCrypt y generar contraseñas válidas de forma automática cumpliendo con los mismos criterios.
Repositorios.Interfaces	IRepositorio	Define las operaciones básicas que debe implementar cualquier repositorio del sistema. Permite agregar un objeto, obtenerlo por su identificador, eliminarlo y recuperar la lista completa de objetos almacenados.

Repositorios.Interfaces	IRepositorioUsuarios	Define las operaciones que cualquier repositorio de usuarios tiene que implementar: buscar por email y actualizar la contraseña encriptada, además de heredar los métodos genéricos del IRepositorio.
Repositorios	RepositorioUsuarios	Implementa los métodos de la interfaz de repositorio de usuarios.
Repositorios	RepositorioProyectos	Implementa los métodos de la interfaz de repositorio.
Repositorios	RepositorioRecursos	Implementa los métodos de la interfaz de repositorio.

Diagrama de paquetes: (Ver anexo 1.1)

El sistema se organiza en paquetes que reflejan una arquitectura modular basada en la separación de responsabilidades. Esta estructura facilita la comprensión, el mantenimiento y la evolución del sistema, alineándose con los principios enseñados en el curso.

El paquete Dominio contiene las entidades centrales del modelo, como usuario, tarea, recurso, proyecto, entre otras. Cada clase encapsula sus propias validaciones, reglas de negocio y comportamientos asociados, promoviendo una alta cohesión. Este paquete no depende de ningún otro y es reutilizado por el resto de los componentes.

El paquete Servicios implementa la lógica de aplicación. Contiene los gestores encargados de orquestar operaciones más complejas sobre las entidades del dominio, garantizando que se respeten los permisos y restricciones definidas. Además, incluye utilidades como CaminoCritico y UtilidadesContrasena.

El paquete Repositorios proporciona almacenamiento en memoria para las entidades, simulando una capa de persistencia. Aunque la dependencia hacia clases concretas limita la flexibilidad, permite cumplir con los requerimientos actuales sin necesidad de una base de datos.

El paquete Interfaz representa la capa de presentación construida en Blazor Server. Interactúa directamente con los gestores del paquete Servicios para obtener y modificar datos del sistema. Esta capa se mantiene libre de lógica de negocio, delegándola completamente a los servicios.

Por último, el paquete Tests contiene las pruebas automatizadas construidas bajo TDD. Estas ejercitan tanto al dominio como a los servicios, y permiten validar el correcto funcionamiento de las reglas de negocio implementadas.

Diagrama de Clases:

- Dominio: (Ver anexo 1.2)

El diagrama de clases del paquete Dominio fue modelado respetando los principios de diseño orientado a objetos enseñados en el curso, como responsabilidad única, alta cohesión, bajo acoplamiento, y siguiendo el patrón GRASP – Information Expert, delegando la lógica a las clases que poseen la información necesaria para ejecutarla.

Se decidió que la clase Proyecto mantuviera la agregación de las tareas con navegabilidad desde proyecto a tarea sea *, reflejando que un proyecto puede existir sin tareas al momento de su creación y puede tener múltiples tareas. Sin embargo, las tareas no pueden existir sin un proyecto que las contenga. Considerando que las tareas dependen conceptualmente de un único proyecto, se decidió que la relación sea de composición y su navegabilidad desde tarea a proyecto sea 1, ya que se entendió que su eliminación debería ocurrir automáticamente si el proyecto desaparece.

La clase Proyecto mantiene una relación directa con Usuario, ya que los miembros del proyecto se modelan como una lista de usuarios. Por eso, se optó por una asociación unidireccional desde Proyecto hacia Usuario, ya que el proyecto necesita conocer a sus miembros, pero un usuario no necesita conocer explícitamente todos los proyectos donde participa. La relación con el administrador también es directa, porque se trata de un usuario dentro de la misma lista, con un rol destacado. Esto justifica que la flecha sea navegable solo desde Proyecto. La cardinalidad de Proyecto a Usuario es 1..* debido a que un proyecto siempre tiene al menos un usuario cuando se crea, que es el administrador de proyecto, aunque también puede tener varios miembros. En cambio, de Usuario a Proyecto es 0..* debido a que un usuario al crearse no es miembro de ningún proyecto pero puede serlo de varios.

La clase Tarea mantiene una relación unidireccional hacia Usuario, representando a los miembros asignados a dicha tarea. Se decidió modelarla como una lista simple, sin clase intermedia, ya que no hay información adicional asociada a la asignación (como fecha de inicio, horas trabajadas, etc.). La cardinalidad desde Tarea hacia Usuario es de 0..*, ya que

una tarea puede no tener usuarios asignados inicialmente, pero permite múltiples asignaciones. Desde el lado de Usuario no se necesita conocer explícitamente las tareas asignadas, por lo tanto, no se definió una relación inversa. En cuanto a la cardinalidad desde Usuario a Tarea es de 0..* ya que un usuario puede no tener tareas asignadas o puede tener varias tareas asignadas.

En cuanto a la relación entre Tarea y Recurso, se siguió un criterio similar. Cada tarea mantiene una lista de recursos necesarios para su ejecución. Esta relación es unidireccional desde Tarea hacia Recurso, con cardinalidad 0..*, ya que una tarea puede ejecutarse sin recursos o requerir varios. No se definió la relación inversa porque un recurso no necesita tener conocimiento directo de las tareas que lo utilizan, ya que su uso se refleja indirectamente mediante la propiedad CantidadDeTareasUsandolo. La cardinalidad desde Recurso a Tarea es de 0..* ya que un recurso puede estar disponible y no ser utilizado por ninguna tarea, o puede ser utilizado por múltiples tareas. Esta decisión se tomó a partir de la ambigüedad presente en la consigna y en las respuestas del foro respecto al uso de recursos, asumiendo que todos los recursos pueden ser utilizados en paralelo por diferentes tareas.

La relación entre Tarea y Dependencia fue modelada como una composición, ya que cada dependencia existe únicamente dentro del contexto de una tarea. Una dependencia representa un vínculo con otra tarea previa del proyecto y especifica un tipo (“FS” o “SS”). Se definió que una tarea puede tener 0..* dependencias (cardinalidad hacia Dependencia), mientras que cada Dependencia debe estar asociada a una única Tarea origen (1). Por ello, la flecha desde Dependencia hacia Tarea tiene cardinalidad 1.

El atributo Estado de Tarea se basa en un enumerado EstadoTarea, que representa los distintos estados posibles: Pendiente, En Proceso, Bloqueada y Completada. Se optó por una enumeración para mejorar la legibilidad, restringir los valores válidos y centralizar la definición de estados. Esta decisión también simplifica las validaciones internas y permite un código más mantenible y expresivo. Como Tarea posee un atributo de tipo EstadoTarea, se definió una relación de asociación entre ambas clases. Debido que Tarea solo puede tener un estado, la cardinalidad desde Tarea a EstadoTarea es 1. Por otro lado la cardinalidad desde EstadoTarea hacia Tarea es de *, ya que puede suceder que ninguna de las tareas del sistema tengan alguno de los estados y que varias tareas tengan algún estado.

Recurso tiene un atributo “ProyectoAsociado” de tipo Proyecto para indicar si el recurso es exclusivo de un proyecto o es general del sistema (en ese caso el proyecto asociado

es null). Por ende se crea una relación de asociación desde Recurso a Proyecto para que Recurso pueda conocer a la clase Proyecto. La cardinalidad desde Recurso a Proyecto es 0..1 por lo mencionado anteriormente. A su vez, la cardinalidad de Proyecto a Recurso es * ya que un proyecto puede no tener ningún recurso exclusivo así como tener varios.

Finalmente, Usuario contiene una lista de Notificacion, reflejando una relación de composición, ya que las notificaciones dependen exclusivamente del usuario que las contiene, y deben ser eliminadas junto con él. Esto refuerza la idea de propiedad fuerte y gestión local de las notificaciones dentro del objeto usuario.

- Servicios: (Ver anexo 1.3)

El diagrama de clases del paquete Servicios fue modelado con el objetivo de centralizar la lógica de aplicación que coordina múltiples entidades del dominio, siguiendo el principio de separación de responsabilidades. Cada clase gestora encapsula operaciones de mayor nivel que no pertenecen directamente a las entidades del dominio, respetando las restricciones de negocio planteadas en la consigna.

El gestor GestorUsuarios se encarga de manejar el ciclo de vida de los usuarios. Agrupa funcionalidades como creación, eliminación, autenticación, asignación de roles y gestión de contraseñas. Para esto último, hace uso de la clase estática UtilidadesContrasena, llamando a sus métodos para validar el formato, encriptar o autogenerar contraseñas. Por tratarse de llamadas a métodos estáticos, la dependencia no se expresa como atributo pero se representa con una relación de dependencia hacia UtilidadesContrasena, indicando un vínculo de uso.

Por otro lado, GestorRecursos se encarga de administrar los recursos del sistema, tanto generales como exclusivos de un proyecto. Controla su creación, modificación y eliminación, validando que el usuario tenga los permisos adecuados para operar sobre el recurso en cuestión. Si se trata de un recurso exclusivo, se verifica también que pertenezca al proyecto que administra el solicitante. Para realizar estas validaciones y obtener el proyecto correspondiente, GestorRecursos mantiene una relación directa con GestorProyectos, representada en el diagrama como una asociación. La relación es unidireccional, desde GestorRecursos hacia GestorProyectos, ya que solo el primero depende del segundo. La cardinalidad de la relación es 1 a 1, ya que cada instancia de GestorRecursos necesita contar

con una única instancia de GestorProyectos para funcionar correctamente, y esa dependencia se inyecta en el constructor y se mantiene durante toda su vida útil.

Dentro del mismo paquete, se encuentra el componente CaminoCritico, definido como clase estática bajo el estereotipo <<utilidades>>. Esta clase encapsula el algoritmo CPM, utilizado para calcular las fechas de inicio/fin más tempranas de las tareas, detectar ciclos en las dependencias mediante ordenamiento topológico, y asignar holguras. Su uso no está limitado a una única clase: tanto GestorTareas como GestorProyectos mantienen una dependencia hacia CaminoCritico, ya que son responsables de invocar el cálculo del camino crítico en contextos distintos. GestorTareas lo hace al modificar, agregar o eliminar tareas, o al gestionar dependencias; mientras que GestorProyectos lo invoca al cambiar la fecha de inicio del proyecto. Por tratarse de métodos estáticos, la relación se representa mediante una flecha punteada unidireccional, indicando una dependencia de uso sin instanciar directamente.

Por último, la clase GestorTareas tiene como responsabilidad gestionar tareas dentro de los proyectos. Fue diseñada para contener operaciones como agregar, eliminar o modificar tareas, administrar sus dependencias, asignar usuarios o recursos, y cambiar su estado. Además, se encarga de recalcular el camino crítico luego de cada operación relevante. Esta clase contiene un atributo de tipo GestorProyectos, ya que muchas validaciones requieren acceder al proyecto correspondiente o verificar roles del solicitante. Por ello tienen una relación de asociación desde GestorTareas hacia Gestor Proyectos.

- Repositorios: (Ver anexo 1.4)

El paquete Repositorios fue diseñado para encargarse del almacenamiento en memoria de las entidades principales del sistema (Usuario, Proyecto y Recurso). Su estructura permite simular una capa de persistencia simple, alineada con el alcance de esta primera etapa del obligatorio, en la que no se requiere uso de bases de datos.

Se definió una interfaz genérica IRepositoryo<T> que establece las operaciones básicas que debe cumplir cualquier repositorio del sistema: agregar, eliminar, buscar por ID y obtener todos los objetos almacenados. Esta interfaz permite estandarizar el comportamiento esperado de todos los repositorios y facilitar una futura extensión a otros tipos de entidades o a un almacenamiento real.

La clase `IRepositorioUsuarios` extiende esta lógica básica y define un contrato específico para usuarios, incluyendo un método adicional para buscar por email (`ObtenerUsuarioPorEmail`). Esta especialización se justifica porque la búsqueda por email es un requerimiento funcional exclusivo del manejo de usuarios, y no se aplica a otras entidades del sistema. La flecha que une `IRepositorioUsuarios` con `IRepositorio` representa herencia entre interfaces, con una relación de generalización.

Cada repositorio mantiene una lista interna (`List<T>`) con los objetos que administra, así como un contador (`cantidadUsuarios`, `cantidadProyectos`, etc.) que permite llevar seguimiento del total de instancias creadas, lo que también facilita la generación de identificadores únicos cuando es necesario.

Cada una de las clases concretas (`RepositorioUsuarios`, `RepositorioProyectos` y `RepositorioRecursos`) implementa las operaciones definidas por la interfaz `IRepositorio`.

La decisión de separar los repositorios por tipo de entidad permite mantener la responsabilidad única en cada clase, manteniendo el código más claro, testeable y alineado con los principios de diseño enseñados en el curso. Además, esta estructura facilita la posible sustitución futura de estos repositorios por implementaciones que utilicen bases de datos u otros mecanismos de persistencia, sin afectar la lógica del dominio ni de los servicios.

Dependencias generadas entre Interfaz Gráfica y Lógica de Negocio

En la aplicación, la comunicación entre el Front-end y Back-end se realiza a través de Data Transfer Objects (DTOs). Estos tienen la responsabilidad de dar una primera validación (campos requeridos, campos con largo máximo, etc) y estructurar la información que la interfaz envía a la lógica del negocio. En este proyecto no fueron utilizados los Controllers, ya que no se profundizó sobre los mismos en el curso. Por este motivo, la comunicación se realiza de forma más directa, utilizando DTOs para desacoplar la interfaz de la lógica de negocio y mantener, en lo posible, la separación de responsabilidades. Esto también explica por qué el paquete Interfaz conoce directamente al paquete Dominio, lo cual puede afectar la mantenibilidad y extensibilidad en ciertos casos.

Cabe recalcar que para el proyecto se implementó una paquete independiente de

servicios, la cual incluye a los gestores, los cuales son cruciales en la lógica del negocio (por ejemplo, validando qué usuarios pueden acceder a cierta información y cuales otros no). Estos gestores también forman parte de la comunicación entre el Front y Back-end.

Cuando el usuario realiza una acción en la interfaz, se construye un DTO con los datos correspondientes. Este DTO se envía a un gestor (en la capa de servicios), que valida la información, consulta o modifica entidades del dominio, y devuelve una respuesta estructurada que la interfaz utiliza para actualizar la vista.

Explicación de validaciones de negocio

Las validaciones de negocio se encuentran principalmente en los gestores, donde se concentra la lógica relacionada con permisos, roles y reglas del sistema. Un ejemplo claro de esto es la clase `GestorProyectos`, en la cual se aplican verificaciones específicas para asegurar que solo determinados usuarios, según su rol, puedan ejecutar ciertas acciones dentro de la aplicación.

En esta clase, los métodos responsables de crear o eliminar proyectos, así como de modificar su nombre, descripción o fecha de inicio, y de gestionar miembros (agregar, eliminar o cambiar al administrador), invocan funciones auxiliares de validación que comprueban si el usuario solicitante tiene los permisos adecuados. Estas funciones reflejan decisiones tomadas por el equipo en base a la lógica del negocio diseñada, definiendo qué acciones están permitidas para cada tipo de rol.

A continuación, se detalla la lógica específica aplicada en este gestor:

- Crear un proyecto: Solo puede crear un proyecto un administrador de proyecto sin proyecto asignado aún, ya que un usuario solo puede administrar un proyecto a la vez.
- Eliminar un proyecto: Solo puede eliminar un proyecto el administrador del mismo.
- Modificar nombre, descripción y fecha de inicio de un proyecto: Solo lo puede hacer un administrador del proyecto.
- Cambiar administrador de proyecto: Solo lo puede cambiar un administrador del sistema, y solo puede asignarse a un miembro del proyecto que tenga el rol de administrador de proyecto y no esté administrando ningún otro proyecto.
- Agregar miembro a proyecto: Solo lo puede hacer un administrador del proyecto.

- Eliminar miembro de un proyecto: Solo un administrador de ese proyecto puede eliminar a un miembro del mismo, que no tenga ninguna tarea asignada.

Por otro lado, en el Gestor de Recursos se maneja toda la lógica de los recursos generales y exclusivos. En este, se encuentran validaciones en:

- Agregar recurso: Solo puede agregar un recurso general un administrador de sistema, y un recurso exclusivo de un proyecto específico el administrador de ese proyecto.
- Eliminar recurso: Solo puede eliminar un recurso general un administrador del sistema, y un recurso exclusivo un administrador de sistema o el administrador del proyecto de ese recurso exclusivo. Si el recurso está en uso, independientemente del si es exclusivo o no, no se permite eliminarlo. Que un recurso esté en uso significa que existe al menos una tarea que lo necesita y no está completada.
- Modificar nombre, tipo y descripción de un recurso: Sólo un administrador del sistema puede modificar un recurso general, en cambio los recursos exclusivos pueden ser modificados tanto por administradores del sistema como por los del proyecto asociado al recurso.

En el Gestor de Tareas también se pueden encontrar validaciones del negocio claves para la lógica del mismo. Las mismas se ven en los métodos de:

- Agregar tarea a un proyecto: Solo lo puede realizar el administrador de ese proyecto, siempre y cuando la fecha de inicio no sea anterior a la del inicio del proyecto.
- Eliminar tarea de un proyecto: Solo lo puede realizar el administrador de ese proyecto, y sólo si la tarea no tiene tareas que dependan de ella.
- Modificar título, descripción, duración y fecha de inicio de la tarea: Solo lo puede realizar el administrador de ese proyecto.
- Cambiar el estado de la tarea: Lo puede hacer cualquier miembro del proyecto. Solo puede cambiarse a “En Proceso” o “Completada”. Es importante recalcar que es en la clase Tarea donde se verifica a qué estados se puede cambiar. Allí, es donde se establece que solo se puede pasar de una tarea “Pendiente” a “En Proceso” y de este estado a “Completada”, y que se puede pasar de una tarea “Bloqueada” a “Pendiente”. Que una tarea esté bloqueada implica que tiene al menos una dependencia de tipo FS con otras tareas que aún no están completadas o que tenga una dependencia de tipo SS con una tarea que esté bloqueada o pendiente.

- Agregar una dependencia a una tarea: Solo lo puede hacer el administrador del proyecto, siempre que ambas existan dentro del mismo. Si la dependencia forma un ciclo en el camino crítico, no se permite agregarla.
- Eliminar una dependencia de una tarea: Solo lo puede hacer el administrador del proyecto.
- Agregar/Eliminar un miembro a una tarea: Solo lo puede hacer el administrador del proyecto, siempre que el usuario que se desea asignar sea miembro del mismo.
- Agregar recurso a una tarea: Solo lo puede hacer un administrador del proyecto. El recurso a agregar puede ser general o exclusivo de ese proyecto.
- Eliminar recurso de una tarea: Solo lo puede hacer un administrador del proyecto si el recurso está asignado a la tarea.

Finalmente, las validaciones de negocio se pueden encontrar en el Gestor de Usuarios:

- Agregar usuario al sistema: Lo pueden realizar solo los administradores de sistema.
- Eliminar usuario al sistema: Lo pueden realizar los propios usuarios a ellos mismos, siempre y cuando no sea el primer administrador de sistema. O un administrador del sistema a cualquier usuario que no sea administrador de sistema. Lo pueden realizar siempre que el usuario no pertenezca a ningún proyecto.
- Dar roles de administrador de sistema y/o administrador de proyecto: Solo lo pueden hacer los administradores del sistema.
- Desasignar administrador de proyecto: Lo pueden hacer los administradores de sistema siempre y cuando el usuario tenga el rol de administrador de proyecto y no esté actualmente administrando uno.
- Reiniciar contraseña: Lo puede hacer el propio usuario a sí mismo, o administradores de sistema para a usuarios que no sean administradores de sistema.
- Autogenerar contraseña: Lo pueden hacer administradores del sistema a otro usuario siempre y cuando no sea administrador de sistema.
- Modificar contraseña: Lo puede hacer el propio usuario a sí mismo, o administradores del sistema para otros usuarios que no sean administradores de sistema.
- LogIn: Lo pueden hacer todos los usuarios, siempre y cuando el email esté registrado y la contraseña sea correcta.

Si no se cumple alguna de las validaciones especificadas, se lanza una excepción de Servicio y en el caso de los estados en Tarea una excepción de Dominio.

En líneas generales, los administradores de sistema son quienes gestionan los permisos y administran a los usuarios. Tienen acceso a todos los proyectos del sistema, aunque si no forman parte de un proyecto como miembro o administrador, solo pueden visualizarlo sin modificarlo. En cuanto a los proyectos, lo único que pueden modificar es el administrador asignado, siempre que el nuevo administrador propuesto sea un miembro del proyecto, tenga el rol adecuado y no esté administrando otro. También tienen control total sobre los recursos generales: pueden crearlos, modificarlos o eliminarlos. Respecto a los recursos exclusivos, no pueden crearlos, pero sí modificarlos o eliminarlos, incluso si pertenecen a un proyecto ajeno. Por otro lado, los administradores de proyecto son responsables de gestionar su propio proyecto, incluyendo la administración de tareas, miembros y recursos exclusivos. Esta separación de responsabilidades permite mantener reglas claras y consistentes sobre qué operaciones puede realizar cada tipo de usuario.

Mecanismos generales y descripción de las principales decisiones de diseño tomadas

Para asignar responsabilidades, se procuró que cada clase tuviera un rol claro y definido, organizando el sistema en capas bien separadas.

En la capa de Dominio se definieron las entidades principales del sistema (Usuario, Tarea, Proyecto, etc.), junto con las excepciones de dominio, que representan errores de negocio específicos. Esta capa se mantuvo libre de dependencias externas y encapsula el conocimiento central del sistema.

La capa de Servicios se estructuró con varias subcarpetas. Por un lado, los gestores son los encargados de aplicar la lógica de negocio, validar acciones según los roles y estados del sistema, y coordinar operaciones entre entidades. Además, se incluyeron las excepciones de servicio, utilizadas para representar principalmente errores de acceso o validaciones que no corresponden al dominio. Por último, se creó una carpeta de utilidades, donde se agrupan funcionalidades transversales como el cálculo del camino crítico (CaminoCritico) y la validación y encriptación de contraseñas (UtilidadesContrasena). Es importante recalcar que para calcular el camino crítico se tomó la decisión de manejar las fechas en días y no en horas.

La capa de Repositorios se encarga del almacenamiento en memoria, manteniendo listas de proyectos, recursos y usuarios, simulando una capa de persistencia que puede ser reemplazada fácilmente por una implementación basada en base de datos en el futuro.

En la capa de Interfaz se encuentran los DTOs, que estructuran los datos que se transfieren entre el backend y el frontend, y los servicios de interfaz, como la lógica de login, que interactúan directamente con los gestores. También se incluyen los componentes Blazor, organizados en carpetas Layout y Pages, que componen la interfaz visual con la que interactúa el usuario final.

Finalmente, la carpeta Tests se organizó en subcarpetas (DominioTests, RepositorioTests, ServiciosTests) según el tipo de prueba, lo que permitió aplicar el enfoque de desarrollo guiado por pruebas (TDD) en la implementación de la lógica del sistema.

Se buscó que cada componente tuviera una única responsabilidad, facilitando así la escalabilidad, el mantenimiento y el testeado del sistema. No obstante, se identificaron oportunidades de mejora. Durante el desarrollo surgieron ideas como encapsular ciertas validaciones comunes en clases auxiliares, implementar un servicio específico de notificaciones y definir excepciones más concretas y especializadas. Estas posibles mejoras serán consideradas para futuras versiones del sistema.

Mantenibilidad y extensibilidad

La arquitectura del sistema fue diseñada priorizando la mantenibilidad y la extensibilidad, de forma que futuras modificaciones o nuevas funcionalidades puedan incorporarse con el menor impacto posible sobre el resto del sistema.

Esto se refleja claramente en la estructura de los repositorios, donde se definieron interfaces (`IRepositorio<T>`, `IRepositorioUsuarios`) que permiten cambiar la implementación del almacenamiento. Por ejemplo, reemplazar la versión en memoria por una basada en base de datos, sin modificar la lógica del dominio ni los servicios. Si bien en esta etapa los gestores utilizan directamente las clases concretas de los repositorios, el hecho de contar con interfaces definidas deja preparada la arquitectura para aplicar inyección de dependencias o mocks en etapas futuras.

También se tomó una decisión importante al abstraer la lógica de validación y encriptación de contraseñas en una clase utilitaria (`UtilidadesContrasena`). Esta clase concentra las reglas de formato, el encriptado con BCrypt y la generación de contraseñas

válidas. Gracias a esto, cualquier cambio en el algoritmo de encriptación puede aplicarse en un solo lugar, sin afectar la lógica de los gestores que la consumen, lo cual mejora la reutilización y reduce errores.

Como mejora a futuro, se identificó que algunas validaciones de roles y permisos se repiten en distintos gestores. Una posible refactorización consistiría en agrupar esa lógica compartida en una clase específica de autorización, lo que permitiría reducir la duplicación, mejorar la cohesión y favorecer aún más el desacoplamiento.

Algoritmo de Camino Crítico y evidencia de TDD: (Ver anexo 2)

Cobertura de pruebas unitarias: (Ver anexo 3)

Evidencia de TDD en la clase Tarea: (Ver anexo 4)

Casos de prueba elaborados

Se evidencian los resultados de ejecutar casos de prueba funcionales para las funcionalidades de Gestión de Usuarios (ABM) en la siguiente lista de reproducción: [Gestión de Usuarios \(ABM\) - pruebas funcionales](#)

En la descripción correspondiente a cada uno de los videos se detallan los aspectos evaluados. Se pone especial énfasis en las pruebas asociadas al alta, baja y modificación de usuarios por parte del administrador del sistema, procurando mantener rigurosidad y exhaustividad en la validación de dichas operaciones. Asimismo, se incluyen pruebas representativas de las acciones habilitadas para los roles de administrador de proyecto y miembro de proyecto, con el fin de evidenciar el correcto funcionamiento de los permisos asignados a cada rol.

Referencias Bibliográficas

Hale, C. (2010, 31 de enero). *How to safely store a password*. Recuperado de <https://codahale.com/how-to-safely-store-a-password/>

IngSoft-DA1. (s.f.). *Ejemplos-tecnologia* [Software]. GitHub. <https://github.com/IngSoft-DA1/Ejemplos-tecnologia>

ProjectManager. (s.f.). *Critical Path Method (CPM) in Project Management*. Recuperado de <https://www.projectmanager.com/guides/critical-path-method>

Chat GPT:

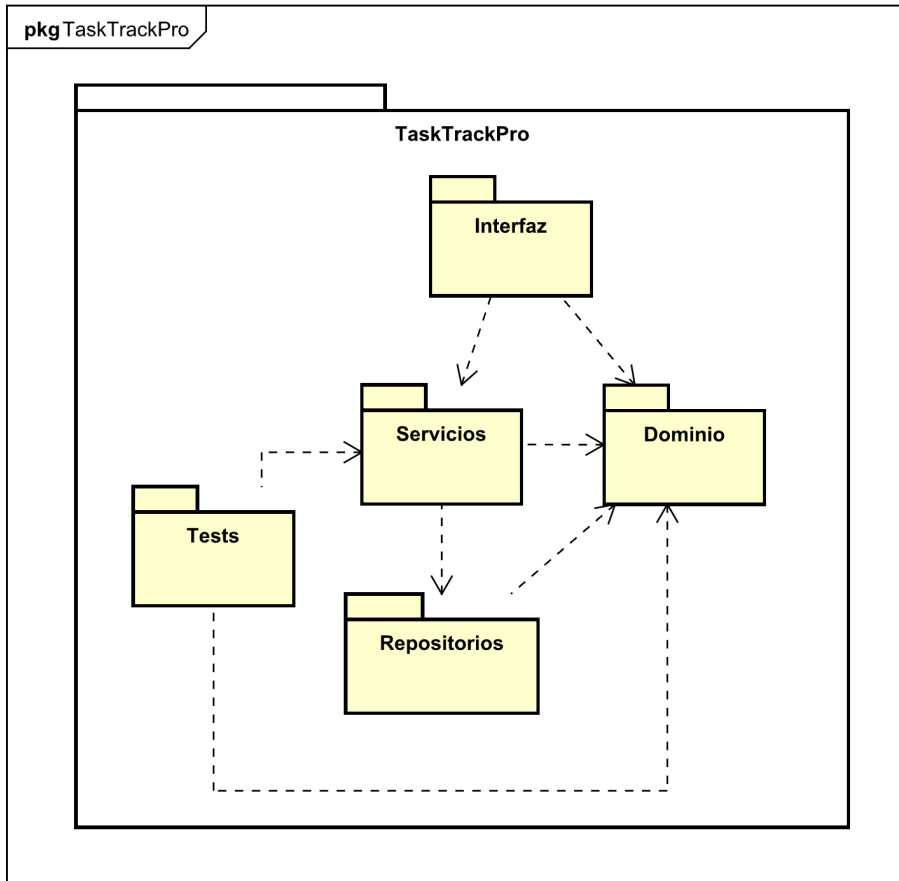
- Cómo implementar el diagrama de Gantt con Oracle APEX.
- Cómo implementar el algoritmo de Camino Crítico
(<https://chatgpt.com/share/67f59b0a-ae9c-8000-aca7-71eb8cf609f7>)

ANEXO 1

Diagramas UML

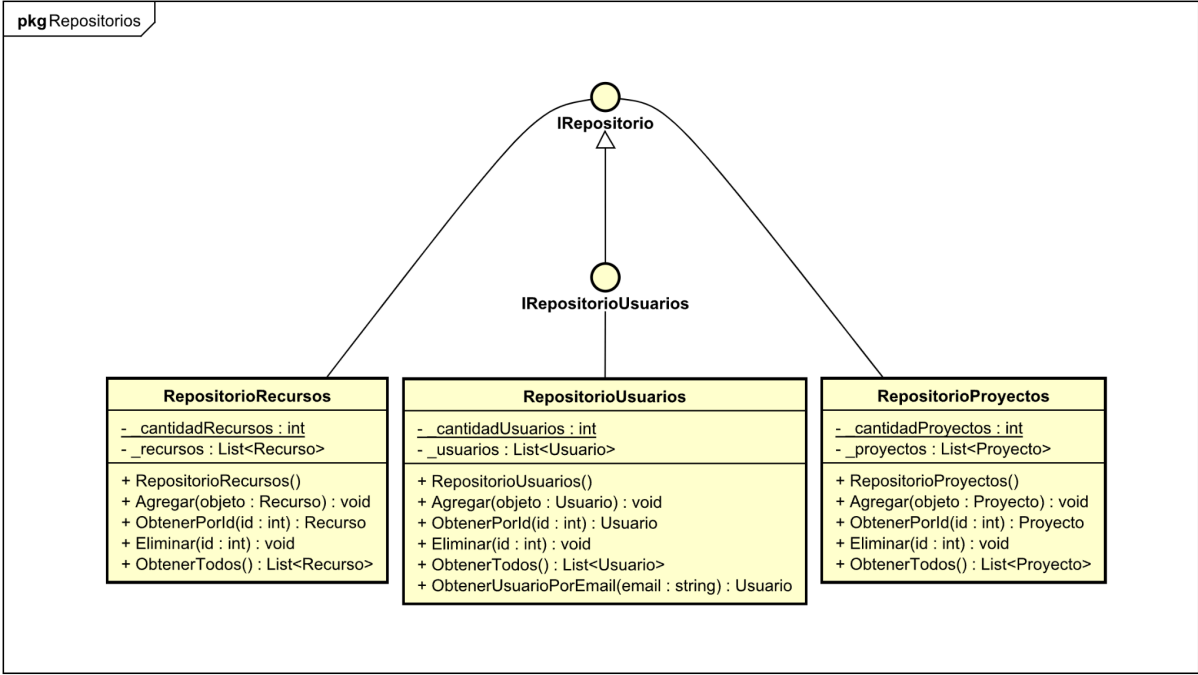
En esta sección del anexo se muestran fotos de los UML, en la carpeta entregada están adjuntados los .svg de cada uno de los diagramas.

1.1 Diagrama de paquetes



[illegible][illegible]

1.4 Repositorios



ANEXO 2

Algoritmo de Camino Crítico y evidencia de TDD

Con el fin de determinar el camino crítico dentro del proyecto, y considerando el alcance funcional de la aplicación desarrollada, fue fundamental identificar en qué momentos del ciclo del proyecto debía recalcularse el camino crítico, considerando las tareas existentes y sus relaciones de dependencia. Esta estimación permite optimizar la gestión del proyecto al identificar aquellas tareas que impactan directamente en su duración total.

Cada tarea del proyecto posee una duración, una fecha estimada de inicio y, en muchos casos, dependencias con otras tareas. Para poder calcular correctamente el camino crítico, es imprescindible gestionar estos datos en conjunto con la fecha de inicio global del proyecto, a fin de determinar la fecha de finalización estimada para cada tarea, así como su holgura, es decir, la cantidad de días que puede retrasarse sin afectar la duración total del proyecto. Además, a partir de este análisis, se puede estimar también la fecha de finalización del proyecto completo.

Un aspecto relevante considerado por el equipo de desarrollo, en función de los requerimientos del cliente, es que la fecha de inicio de una tarea debe considerarse estimada y sujeta a cambio. En particular, si las dependencias de una tarea lo permiten, esta podrá iniciarse antes de lo previsto originalmente, ajustándose su programación en consecuencia. Este enfoque dinámico busca reflejar una planificación más realista y flexible, alineada con las condiciones reales del proyecto.

Entonces, los escenarios en los que es necesario calcular la ruta crítica del proyecto son los detallados a continuación:

- Cambios generales del proyecto:
- Después de modificar la fecha de inicio del proyecto.
- Después de agregar una tarea al proyecto.
- Después de eliminar una tarea del proyecto.
- Después de modificar la duración de una tarea.
- Después de modificar la fecha de inicio de una tarea.
- Después de cambiar el estado de una tarea.
- Antes de agregar una dependencia a una tarea, para evitar dependencias cíclicas.
- Después de eliminar una dependencia de una tarea.
- Antes de visualizar el diagrama de Gantt de un proyecto.

El análisis realizado permitió definir la mejor forma para implementar el cálculo del camino crítico buscando alta cohesión y bajo acoplamiento.

En primer lugar, se definió que en el cálculo del camino crítico solamente interactúan las clases Tarea y Proyecto, dado que son las entidades que se deben analizar y modificar.

En segundo lugar, se vio que las clases que deberían hacer uso del algoritmo son los gestores, al realizar cambios sobre las clases de dominio. Por tanto, GestorTareas y GestorProyectos hacen uso del algoritmo. Adicionalmente, en el paquete interfaz también se hace uso del cálculo del camino crítico, al momento de renderizar la ventana del diagrama de Gantt (para asegurar mostrar un camino crítico actualizado con la fecha actual y estado actual del proyecto).

Dadas las consideraciones de interacción entre clases y el algoritmo, se optó por crear una clase de utilidades llamada CaminoCritico, la cual posee el método CalcularCaminoCritico, que dado un proyecto lo modifica así como a sus tareas. Esta clase fue definida como estática dado que su funcionalidad no requiere instancias y su propósito se limita al procesamiento de datos mediante un único algoritmo puro, sin necesidad de mantener estado interno.

Por otro lado, GestorTareas y GestorProyectos dependen de la clase de CaminoCritico ya que hacen uso del método de cálculo.

En términos de cohesión y acoplamiento, se priorizó una alta cohesión dentro de cada clase —donde cada una tiene responsabilidades claramente definidas— y un bajo acoplamiento entre ellas, dado que la clase CaminoCritico actúa como un servicio utilitario reutilizable, minimizando las dependencias directas entre componentes de dominio.

En cuanto a la implementación de los métodos de la clase y el uso de TDD, resulta útil entender correctamente el algoritmo genérico. La forma de identificar un camino crítico se puede dividir en dos pasos: *Forward Pass* y *Backward Pass*. El primero se basa en recorrer las tareas en orden de ejecución para calcular las fechas más tempranas posibles de inicio y fin de cada tarea, considerando sus dependencias, mientras que el segundo paso se basa en recorrer las tareas en orden inverso para calcular las fechas más tardías posibles sin retrasar el proyecto, y así determinar la holgura de cada tarea.

Para poder entender el algoritmo se realizó un pseudocódigo (se detalla al final del presente anexo). Con el objetivo de utilizar TDD, se definieron las funciones auxiliares necesarias, las cuales se implementaron siguiendo el ciclo RED-GREEN-REFACTOR.

A continuación, fue necesario hacer un refactor para cambiar la visibilidad de los métodos, e integrarlos todos en un método `CalcularCaminoCritico`. Luego los tests fueron modificados para probar casos de cálculos de caminos críticos.

Más adelante, fue necesario seguir editando el método para tener en cuenta factores como los estados de la tarea que en un comienzo no fueron considerados.

```
73a51b1 (HEAD -> feature/creacionCaminoCritico, origin/feature/creacionCaminoCritico) REFACTOR: tests extras
bf76e5e REFACTOR: Cambio de tests para testear solo métodos public
e1563b0 REFACTOR: método camino critico
1380d4f GREEN: Se pueden calcular holguras de tareas
47ad38e RED: No se pueden calcular holguras de tareas
54d982f GREEN: Se calculan las fechas de inicio más temprano de una tarea en base a dependencias
5b40956 RED: No se calculan las fechas de inicio más temprano de una tarea en base a dependencias
3f6a5e5 REFACTOR: TestInitialize
b8136f1 GREEN: Se pueden obtener las sucesoras de una tarea
a4c6814 RED: No se puede obtener las sucesoras de una tarea
6c47a23 REFACTOR: método auxiliar
cf0587f GREEN: Se puede obtener el orden topológico de una lista de tareas
6c790cf RED: No se puede obtener el orden topológico de una lista de tareas
```

Pseudocódigo realizado:

función `Calcular(Proyecto proyecto)`:

`tareas = proyecto.Tareas`

`// === Paso 1: Orden topológico (respeta dependencias)`

`ordenadas = ordenarTopologicamente(tareas)`

`// === Paso 2: FORWARD PASS ===`

`para cada tarea en ordenadas:`

`si tarea no tiene dependencias:`

`tarea.FechaInicio = proyecto.FechaInicio // o fecha base`

`sino:`

`fechasInicioPosibles = []`

`para cada dependencia en tarea.Dependencias:`

`tareaPrevia = dependencia.TareaPredecesora`

`si dependencia.Tipo == "FS":`

`fechasInicioPosibles.agregar(tareaPrevia.FechaFinMasTemprana)`

`si dependencia.Tipo == "SS":`

`fechasInicioPosibles.agregar(tareaPrevia.FechaInicio)`

`tarea.FechaInicio = máximo(fechasInicioPosibles)`

```

    tarea.FechaFinMasTemprana = tarea.FechaInicio + tarea.Duracion

// ==== Paso 3: Fecha fin del proyecto ====
proyecto.FechaFinMasTemprana = máximo(t.FechaFinMasTemprana para t en tareas)

// ==== Paso 4: Construir mapa de sucesoras ====
sucesoras = mapearSucesoras(tareas)

// ==== Paso 5: BACKWARD PASS ====
para cada tarea en ordenadas (en orden inverso):
    si tarea no tiene sucesoras:
        fechaLimite = proyecto.FechaFinMasTemprana
    sino:
        fechasLimitePosibles = []

        para cada sucesora en sucesoras[tarea]:
            para cada dependencia en sucesora.Dependencias donde
dependencia.TareaPredecesora == tarea:
                si dependencia.Tipo == "FS":
                    fechasLimitePosibles.agregar(sucesora.FechaInicio)
                si dependencia.Tipo == "SS":
                    fechasLimitePosibles.agregar(sucesora.FechaInicio)

        fechaLimite = mínimo(fechasLimitePosibles)

    tarea.Holgura = fechaLimite - tarea.FechaFinMasTemprana

// ==== Paso 6: Marcar tareas críticas (opcional)
para cada tarea en tareas:
    si tarea.Holgura == 0:
        tarea.EsCritica = true

función ordenarTopologicamente(tareas):

// 1. Inicializar mapa de grados de entrada
mapaGradoEntrada = diccionario<Tarea, int>()
mapaSucesoras = diccionario<Tarea, List<Tarea>>()

para cada tarea en tareas:
    mapaGradoEntrada[tarea] = 0
    mapaSucesoras[tarea] = []

```

```
// 2. Contar grados de entrada y construir mapa de sucesoras  
para cada tarea in tareas:
```

```
    para cada dependencia en tarea.Dependencias:  
        tareaPrevia = dependencia.TareaPredecesora  
        mapaGradoEntrada[tarea] += 1  
        mapaSucesoras[tareaPrevia].agregar(tarea)
```

```
// 3. Inicializar cola con tareas sin dependencias (grado 0)  
cola = lista de tareas donde mapaGradoEntrada[t] == 0  
resultado = lista vacía
```

```
// 4. Procesar tareas en orden  
mientras cola no esté vacía:  
    actual = cola.removePrimero()  
    resultado.agregar(actual)
```

```
    para cada sucesora en mapaSucesoras[actual]:  
        mapaGradoEntrada[sucesora] -= 1  
        si mapaGradoEntrada[sucesora] == 0:  
            cola.agregar(sucesora)
```

```
// 5. Validación final  
si resultado.tamaño != tareas.tamaño:  
    lanzar Excepcion ("El grafo tiene un ciclo")
```

```
retornar resultado
```

función mapearSucesoras(tareas):

```
    sucesoras = diccionario<Tarea, List<Tarea>>()
```

```
// Inicializar el diccionario vacío  
para cada tarea en tareas:  
    sucesoras[tarea] = lista vacía
```

```
// Recorrer todas las tareas y sus dependencias  
para cada tarea in tareas:  
    para cada dependencia en tarea.Dependencias:  
        tareaPredecesora = dependencia.TareaPredecesora
```

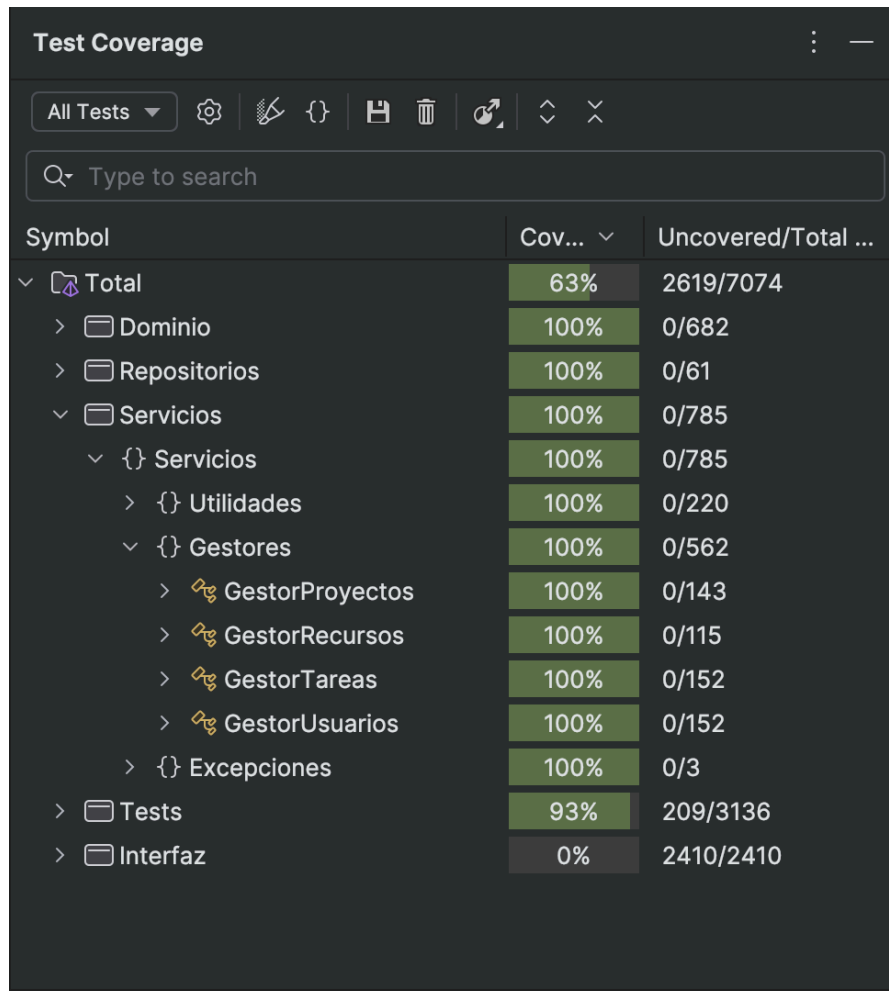
```
        // Agregar la tarea actual como sucesora de su predecesora  
        sucesoras[tareaPredecesora].agregar(tarea)
```

```
retornar sucesoras
```

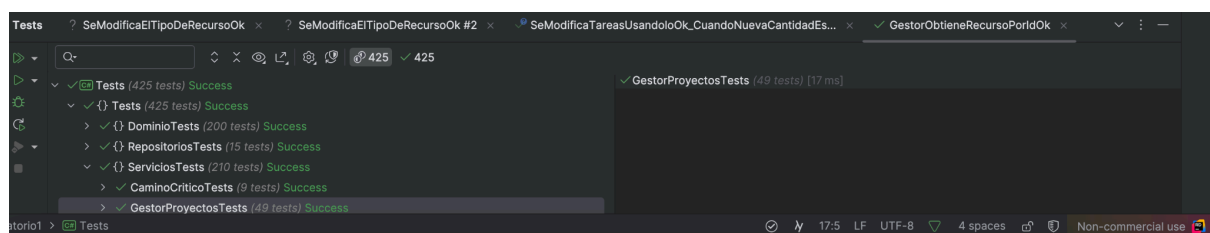
ANEXO 3

Cobertura de pruebas unitarias

En esta sección del anexo se muestran capturas de pantalla de la cobertura de los tests unitarios, en la carpeta entregada está adjuntado el .html de la cobertura.



Symbol	Cov...	Uncovered/Total ...
▼ Total	63%	2619/7074
> Dominio	100%	0/682
> Repositorios	100%	0/61
▼ Servicios	100%	0/785
▼ Servicios	100%	0/785
> Utilidades	100%	0/220
▼ Gestores	100%	0/562
> GestorProyectos	100%	0/143
> GestorRecursos	100%	0/115
> GestorTareas	100%	0/152
> GestorUsuarios	100%	0/152
> Excepciones	100%	0/3
> Tests	93%	209/3136
> Interfaz	0%	2410/2410



ANEXO 4

Evidencia de TDD en la clase Tarea

En esta sección del anexo se muestra captura de pantalla de algunos de los commits, en el repositorio de github, en el historial de commits se evidencia claramente el uso de TDD en su totalidad.

```
b71c59a RED: ModificarDescripcion debe lanzar una excepcion si se da una nueva descripcion null o vacia
1ea4ae5 GREEN: ModificarDescripcion funciona correctamente
f4e5448 RED: No existe el método modificarDescripcion
3acde17 GREEN: Si se quiere modificar un titulo a vacío da excepcion
6b2e2d0 GREEN: Si se quiere modificar un titulo a null da excepcion
517f5af RED: No se puede modificar el titulo a uno null, debería dar una excepcion
f26bfe7 GREEN: El metodo modificarTitulo modifica el titulo correctamente
725d55d RED: No existe el metodo modificarTitulo
53be80d GREEN: Si el recurso no existe se lanza la excepcion correspondiente
2c74eba RED: Debe lanzarse una excepcion si el recurso no existe
cfde971 GREEN: Se elimina un recurso de la lista correctamente
8354b01 RED: No existe Id para recurso, ni metodo EliminarRecurso
30edc7a GREEN: No se puede agregar un recurso ya agregado a recursosNecesarios
77e6c22 RED: Debe tirar una excepcion si se quiere aagregar un recurso ya agregado
4a21d1d GREEN: Tira la excepcion correspondiente si el recurso es null
3af11c1 RED: Debe tirar una excepcion si el recurso es null
11d697b GREEN: La funcion AgregarRecurso agrega un recurso a RecursosNecesarios
c5375bc RED: No existe la funcion AgregarRecurso
941a0ae GREEN: Da la excepción correspondiente si se quiere eliminar un usuario no existente
ddc4bd4 RED: Deberia tirar una excepcion si el usuario a eliminar no existe
b130670 GREEN: El metodo EliminarUsuario elimina un usuario de la lista dado su id
1adb60e RED: No existe el metodo EliminarUsuario
ec370ec GREEN: No se puede agregar un usuario repetido a la lista, tira una exception
48ae7ba RED: Se asignan dos usuarios sin tirar exception a la lista de usuarios asignados
dfd4bd0 GREE: Se tira la excepcion corresopndiente si se quiere asignar un usuario que es null
8cd7b86 RED: No se tiro la excepcion de dominio si el usuario es null
afbe984 GREEN: AsignarUsuario agrega un usuario a la lista
117507b RED: AsignarUsuario no está definida
f9b107f GREEN: Existe el constructor y el atributo Id y se asigna todo correctamente
034ca26 RED: No existe constructor con 5 parametros ni atributo ID
```

ANEXO 5

Librerías externas

Para poder realizar la visualización del diagrama de Gantt, se utilizó una librería externa llamada Oracle APEX, la versión 24.2.0. Para hacer que el diagrama funcione, se debió definir los campos correspondientes al título de la tarea, su fecha de inicio y de fin y su holgura. Este último campo se tomó para poder definir qué tareas participan del camino crítico, y así poder “pintarlas” de color rojo. El resto de las tareas se visualizan de color celeste.

Fuente: [Apex Oracle Sample Charts](#)

Por otro lado, para garantizar la seguridad en el almacenamiento y validación de contraseñas, se optó por utilizar una librería externa que implementa el algoritmo BCrypt. Este algoritmo es reconocido por su seguridad, ya que hace que el proceso de generación del hash sea deliberadamente lento y configurable, dificultando así los ataques por fuerza bruta y el uso de hardware especializado para la ruptura de contraseñas. Además, incorpora automáticamente un valor aleatorio denominado salt, que asegura que incluso contraseñas idénticas generen hashes diferentes, incrementando la protección contra ataques con tablas precomputadas. La librería utilizada fue BCrypt.Net, versión 4.0.3. Para más información, se puede consultar la documentación oficial en: [BCrypt.Net](#).

ANEXO 6

Bosquejo de UI

Se adjunta el enlace a la presentación realizada en Canva, donde se diseñó el bosquejo inicial de la interfaz de usuario. Este diseño fue ajustado y modificado a lo largo del desarrollo, a medida que surgieron nuevos requerimientos o se identificaron aspectos no contemplados en la planificación original.

La presentación incluye propuestas de estilo, como combinaciones de colores y detalles visuales, que no se llegaron a implementar por razones de tiempo. De todas formas, se priorizó el cumplimiento funcional de los requisitos del sistema, enfocándose en brindar una experiencia de usuario clara, fluida y fácil de utilizar. Para ello, se implementaron elementos como alertas, mensajes de éxito o error, y una organización visual que permita al usuario interactuar con el sistema de forma intuitiva.

Igualmente, el bosquejo fue de gran ayuda durante todo el desarrollo, ya que permitió al equipo organizarse, definir con claridad qué debía contener cada ventana y cómo distribuir los elementos. A partir del diseño surgieron debates, ajustes y nuevos métodos que enriquecieron el proceso, y si bien se hicieron cambios, en líneas generales se respetó la estructura planteada en ese esquema inicial.

https://www.canva.com/design/DAGmorToBos/kPtn_jt29vi7Zf0QaZwt1A/edit?utm_content=DAGmorToBos&utm_campaign=designshare&utm_medium=link2&utm_source=sharebutton

ANEXO 7

Datos precargados

Para facilitar la prueba del funcionamiento de la aplicación, se incluye una serie de datos precargados, definidos en el archivo Program.cs, dentro del paquete de interfaz. En esta sección se incluye información básica de esos datos y se proporciona el usuario y la contraseña del administrador inicial del sistema.

Primer administrador de sistema:

- Email: admin@sistema.com
- Contraseña: TaskTrackPro@2025

Usuario que es administrador de sistema y de proyecto a la vez:

- Email: admin@gmail.com
- Contraseña: Admin123\$
- Administra 'Proyecto A'

Usuario solamente con rol de administrador de proyecto:

- Email: adminProyecto@gmail.com
- Contraseña: Contraseña123\$

Usuario sin rol:

- Email: sofia@gmail.com
- Contraseña: Contraseña123\$
- Está asignada a 'Tarea 2'

Elementos de 'Proyecto A':

- Tarea 1
- Tarea 2
- Recurso EXCLUSIVO del Proyecto A

Recurso general:

- Recurso GENERAL